# MoneyWise — Етап 1

**Проект №03: личен бюджет в евро**
Екип: Адам Биков и Реджеб Туджар · 10. клас
Хранилище: *(добавете линк към GitHub)*

Съдържание: Проучване · План · Екрани · ER диаграма · Седмичен отчет с индивидуални отчети

# RESEARCH — MoneyWise

**Екип:** Адам Биков и Реджеб Туджар · **Етап 1** · 10. клас

## 1. Избрана тема

**Проект №03 — MoneyWise: управление на личен бюджет в евро.** Уеб приложение, в което потребителят записва приходи и разходи, определя бюджети и спестовни цели и анализира финансите си чрез месечни отчети и диаграми.

## 2. Какъв проблем решава

Много младежи не знаят къде отиват парите им: джобните и първите заплати „се изпаряват“ към средата на месеца. Съществуващите приложения са или платени, или претрупани, или само на английски, а повечето само *показват* какво е станало, без да казват какво ще се случи. С въвеждането на еврото в България е нужен и инструмент, който работи естествено в евро.

## 3. За кого е предназначен

- **Ученици 8.–12. клас**, които получават джобни или имат първа работа (напр. 150 € на месец) и искат да спестят за конкретна цел — телефон, екскурзия, курс.
- **Млади хора до 25 г.**, които плащат първи наем и сметки и имат нужда от месечен бюджет.
- Родители и учители по финансова грамотност като нагледен учебен инструмент (с демо данни, без лична информация).

## 4. Какво трябва да може крайният продукт

1. Регистрация и вход; всеки потребител вижда само своите данни.
2. Добавяне, редактиране и изтриване на приходи и разходи (с валидация).
3. Категории (готови и собствени).
4. Месечни бюджети по категории с предупреждение преди превишаване.
5. Спестовни цели със срок и вноски.
6. Месечен отчет и диаграми (по категории, паричен поток).
7. **Собствен алгоритъм:** прогноза на остатъка до края на месеца и откриване на превишен бюджет.
8. Български и английски език; адаптивен интерфейс.

## 5. Три сходни решения

### 5.1. YNAB (You Need A Budget) — https://www.ynab.com

- **Основни функционалности:** бюджет по методологията „дай задача на всяко евро“, цели, отчети, синхронизация с банка.

- **Какво ни харесва:** ясна идея — парите се разпределят *преди* да бъдат похарчени; добри отчети.
- **Какво бихме подобрили:** платен и сложен за начинаещ; няма български език; методологията изисква обучение.
- **Идея за нас (със собствено изпълнение):** лимити по категории, но с автоматично предупреждение „при този темп ще надхвърлиш бюджета на 23-о число“ — собствена формула за проекция на темпа.

### 5.2. Wallet by BudgetBakers — https://www.budgetbakers.com

- **Основни функционалности:** няколко сметки, категории, бюджети, цели, графики, мултивалутност.
- **Какво ни харесва:** богата функционалност, удобно мобилно приложение, цели със срок.
- **Какво бихме подобрили:** интерфейсът е претрупан за ученик; част от възможностите са платени; целите не казват *колко да отделяш месечно*.
- **Идея за нас:** спестовни цели със срок + алгоритъм, който разпределя месечния излишък между няколко цели според спешност и приоритет.

### 5.3. Spendee — https://www.spendee.com

- **Основни функционалности:** проследяване на разходи, категории, споделени портфейли, визуални отчети.
- **Какво ни харесва:** красив, ясен интерфейс; цветни категории.
- **Какво бихме подобрили:** безплатната версия е ограничена; няма прогноза и персонални съвети; въвеждането на разход иска няколко стъпки.
- **Идея за нас:** бързо въвеждане с един ред текст („кафе 4.50 вчера“) и автоматично разпознаване на категория — със собствен разпознавач на ключови думи.

*Забележка: описанията са по публично достъпна информация за продуктите; екипът трябва да ги провери на сайтовете преди предаване и да добави собствени наблюдения.*

## 6. Как MoneyWise се различава

| | YNAB | Wallet | Spendee | **MoneyWise** |
|---|---|---|---|---|
| Безплатен | не | частично | частично | **да** |
| Български език | не | частично | не | **да (BG/EN)** |
| Фокус върху евро | не | не | не | **да** |
| Прогноза на остатъка | не | не | не | **да (собствен алгоритъм)** |
| Предупреждение за превишаване по темп | не | не | не | **да** |
| Разпределение на излишъка между цели | не | не | не | **да** |

Проектът **не копира код или дизайн** на тези продукти — ползваме само общи идеи (бюджети, цели, диаграми), а алгоритмите, интерфейсът и базата данни са наши.

# PROJECT_PLAN — MoneyWise

**Екип:** Адам Биков и Реджеб Туджар

## 1. Функции на крайния проект (поне пет)

| # | Функция | Какво прави потребителят |
|---|---|---|
| 1 | Регистрация и вход (Supabase Auth) | създава акаунт, влиза; вижда само своите данни |
| 2 | Добавяне на приходи/разходи | попълва сума, категория, дата, бележка (или бърз ред „кафе 4.50“) |
| 3 | Редактиране и изтриване | коригира грешка, трие запис след потвърждение |
| 4 | Търсене и филтриране | по текст, тип, категория и месец |
| 5 | Бюджети по категории | задава месечен лимит, вижда прогрес и предупреждения |
| 6 | Спестовни цели | задава сума и срок, прави вноски |
| 7 | Статистика и диаграми | месечен отчет, диаграма по категории, паричен поток |
| 8 | Двуезичност | сменя BG/EN без загуба на данни |

## 2. Собствен алгоритъм: прогноза на остатъка и превишен бюджет

**Входни данни:** списък транзакции (сума, тип, дата, категория), бюджети (категория, лимит), днешна дата.
**Обработка (псевдокод):**

```
функция прогноза(транзакции, днес):
    d = ден от месеца;  rem = дни_в_месеца - d
    фиксирани   = разходи от категория „Жилище“
    променливи  = всички останали разходи
    дневни[i]   = сума на променливите за всеки от последните 45 дни
    ограничи всяка стойност до 4 × медиана(ненулевите дневни)   # еднократни големи покупки
    EMA = 0; за всеки ден: EMA = 0.12*ден + 0.88*EMA
    темп = 0.6*EMA + 0.4*(променливи_този_месец / d)
    фиксирани_очаквани = max(фиксирани_този_месец, фиксирани_миналия_месец)
    прогноза_разход = разходи_досега + (фиксирани_очаквани - фиксирани_този_месец) + темп*rem
    очакван_приход  = приходи от миналия месец, които още не са пристигнали
    върни {прогноза_разход, остатък = приходи + очакван_приход - прогноза_разход}

функция превишен_бюджет(похарчено, лимит, днес):
    проекция = похарчено / ден_от_месеца * дни_в_месеца
    ако похарчено > лимит        → "exceeded"
```

```
    иначе ако проекция > лимит   → "risk"      # + ден на изчерпване = [лимит / (похарчено/ден)]
    иначе ако похарчено ≥ 85% лимит → "warning"
    иначе → "ok"
```

**Резултат:** прогнозиран разход и остатък в края на месеца; статус на всеки бюджет и ден на изчерпване; съвети на таблото. Подробно описание, гранични случаи, сложност и ръчни примери: ALGORITHM.md.

## 3. Технологии

- **HTML5, CSS3, JavaScript (ES2020)** — интерфейс и логика, без framework.
- **Supabase** (PostgreSQL, Auth, Row Level Security) — база данни, вход, права за достъп.
- **GitHub + GitHub Pages** — хранилище и публикуване.
- **Three.js** (допълнителна библиотека) — 3D визуализации (монета, 3D диаграма); нужна е, защото WebGL без библиотека е твърде сложен, а 3D анализът е нагледен за ученици. Не е критична за функционалността.
- **supabase-js** — официалният клиент за връзка с Supabase.
- **Chart.js** (препоръчана в заданието) — не ползваме; 2D диаграмите са собствен SVG код, за да имаме пълен контрол и по-малко зависимости. При нужда може да се замени.
- Външни API: **не използваме**.

## 4. Структура на проекта

```
 src/index.html, src/css/style.css, src/js/*.js   (приложението)
database/schema.sql                               (таблици, RLS)
docs/                                             (документация, макети, ER)
tests/algo.test.js                                (автоматични тестове)
```

*(В заданието са посочени index.html, style.css, script.js — при нас логиката е разделена на няколко модула за по-ясна структура.)*

## 5. Екрани, база данни, задачи

SCREENS.md · ER_DIAGRAM.md · ISSUES.md

## 6. Разпределение

| | |
|---|---|
| **Адам Биков** | **интерфейс, навигация, визуализации, адаптивен дизайн, превод BG/EN** |
| **Реджеб Туджар** | **база данни и RLS, собствен алгоритъм, статистика, автоматични тестове** |
| Двамата | проучване, интеграция, Issues, документация, защита |

## 7. План по седмици

| Седм. | Цел |
|---|---|
| 1 | Проучване, план, макети, ER, хранилище, Issues |
| 2 | Екрани + навигация, Supabase таблици, RLS, тестови данни |
| 3 | CRUD, валидация, зареждане/записване, MVP |
| 4 | Алгоритъм, статистика, ≥10 теста, обработка на грешки |
| 5 | Сигурност, мобилен изглед, потребителско тестване, публикуване, презентация |

# Начален дизайн — екрани на MoneyWise

Екраните по-долу са снимки на **реално работещия прототип** (демо данни, без лична информация). Файловете са в `docs/screens/`.

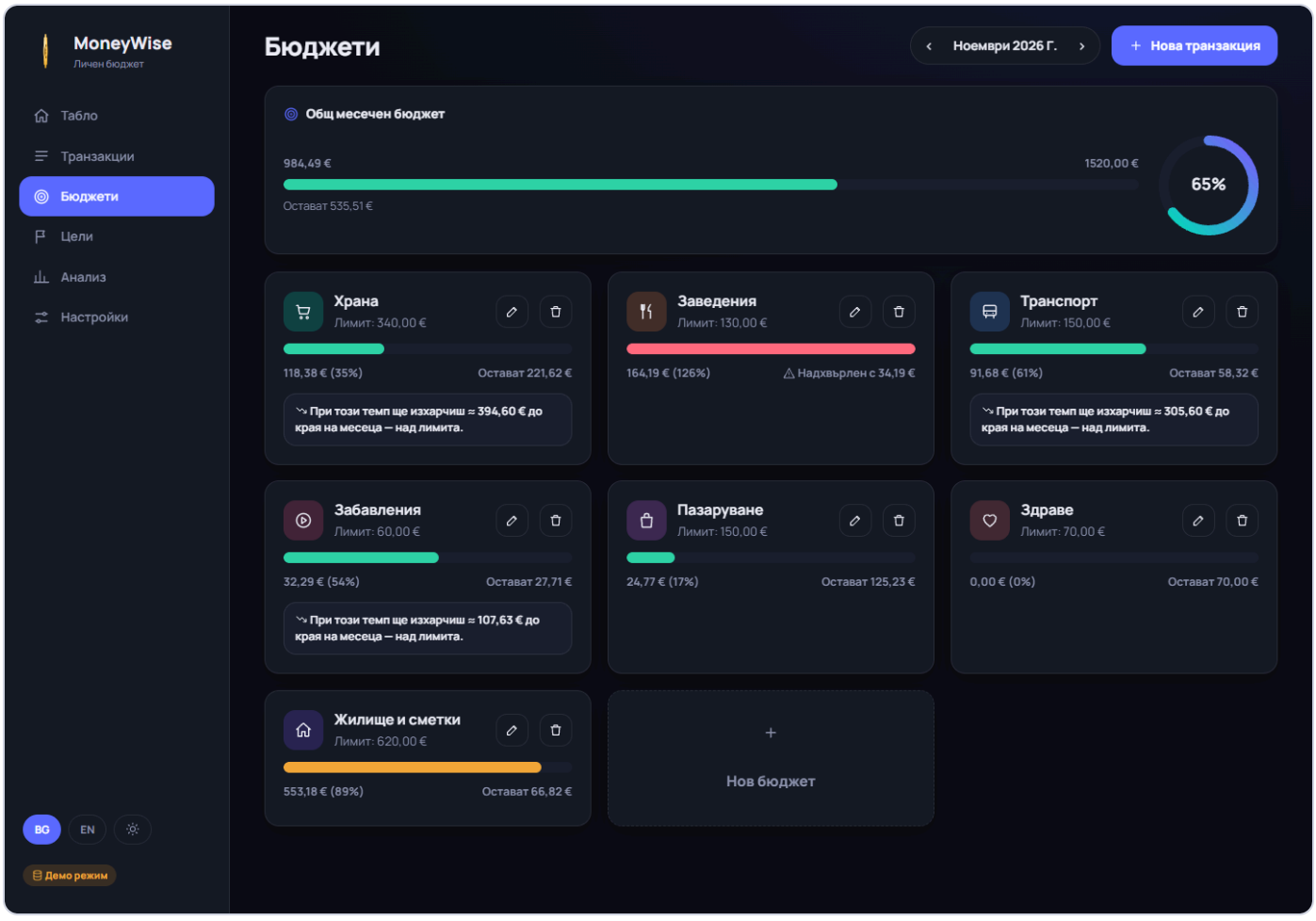
## Екран 1 — Табло (основен работен екран)




| Елемент | Какво прави потребителят |
|---|---|
| Меню вляво (Табло, Транзакции, Бюджети, Цели, Анализ, Настройки) | сменя екрана |
| Избор на месец ‹ › | преглежда друг месец |
| „Нова транзакция“ (клавиш N) | добавя приход или разход |
| Карта с баланс | щраква, за да се обърне и покаже месечно обобщение |

| Елемент | Какво прави потребителят |
|---|---|
| Лента за бързо въвеждане | пише „кафе 4.50“ → създава се разход с автоматична категория |
| Карти Приходи / Разходи / Нетно / Спестяване | следи месеца и сравнението с предходния |
| Диаграма „Паричен поток“ | вижда приходите и разходите за 6 месеца |
| „Финансово здраве“ | вижда оценка 0–100 и от какво се състои |
| 3D монета | украсителен елемент, реагира на мишката |
| BG / EN, тема | сменя езика и светла/тъмна тема |

## Екран 2 — Бюджети

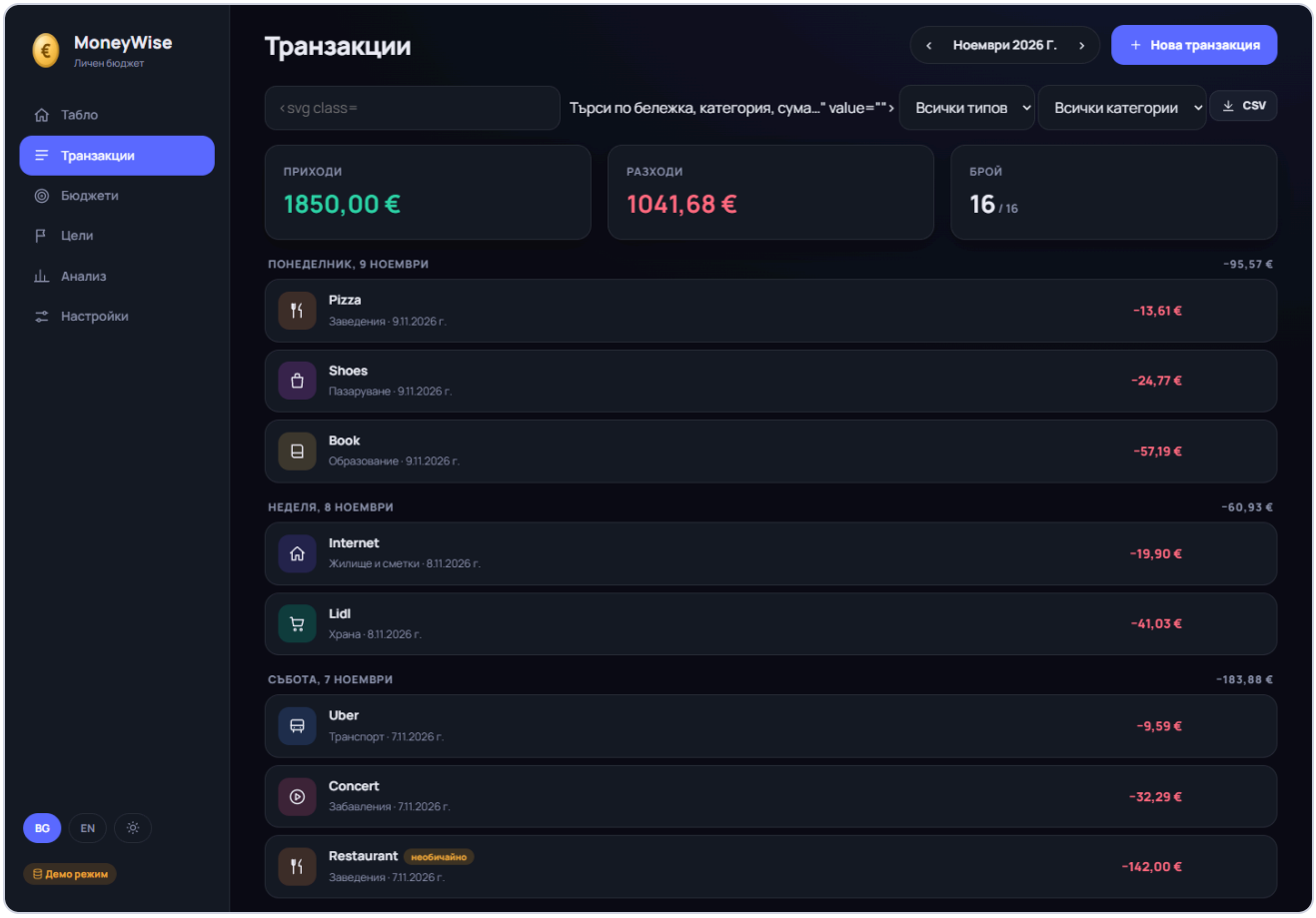



| Елемент | Какво прави потребителят |
|---|---|
| Общ месечен бюджет с пръстен | вижда колко от общия лимит е използвано |

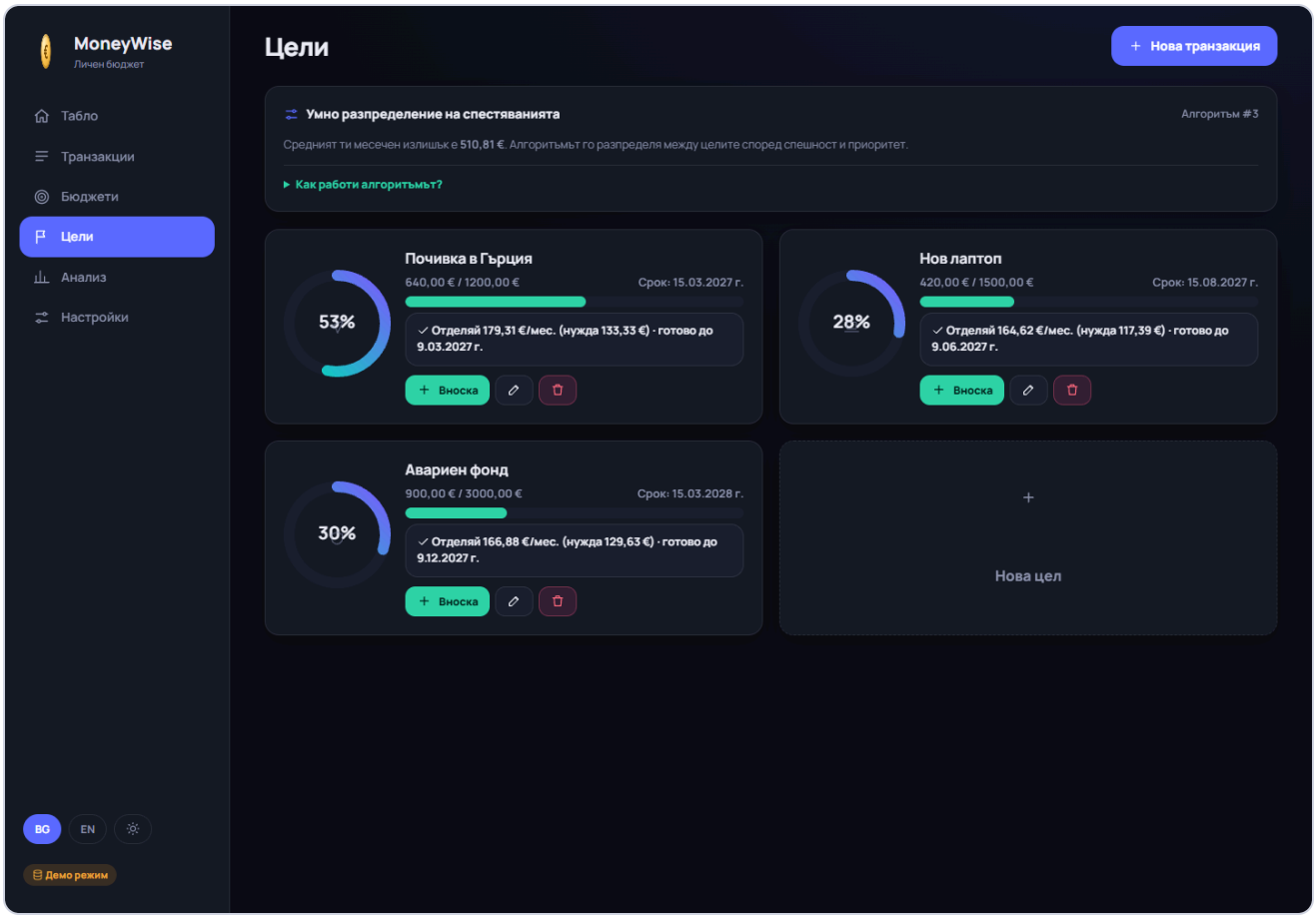
| Елемент | Какво прави потребителят |
|---|---|
| Карта на категория | вижда похарчено / лимит; цветът е зелен → жълт (≥85%) → червен (>100%) |
| Жълта бележка | получава предупреждение „при този темп ще надхвърлиш лимита“ |
| Бутони „молив“ и „кошче“ | редактира или изтрива бюджет |
| „Нов бюджет“ | избира категория и месечен лимит |

## Допълнителни екрани

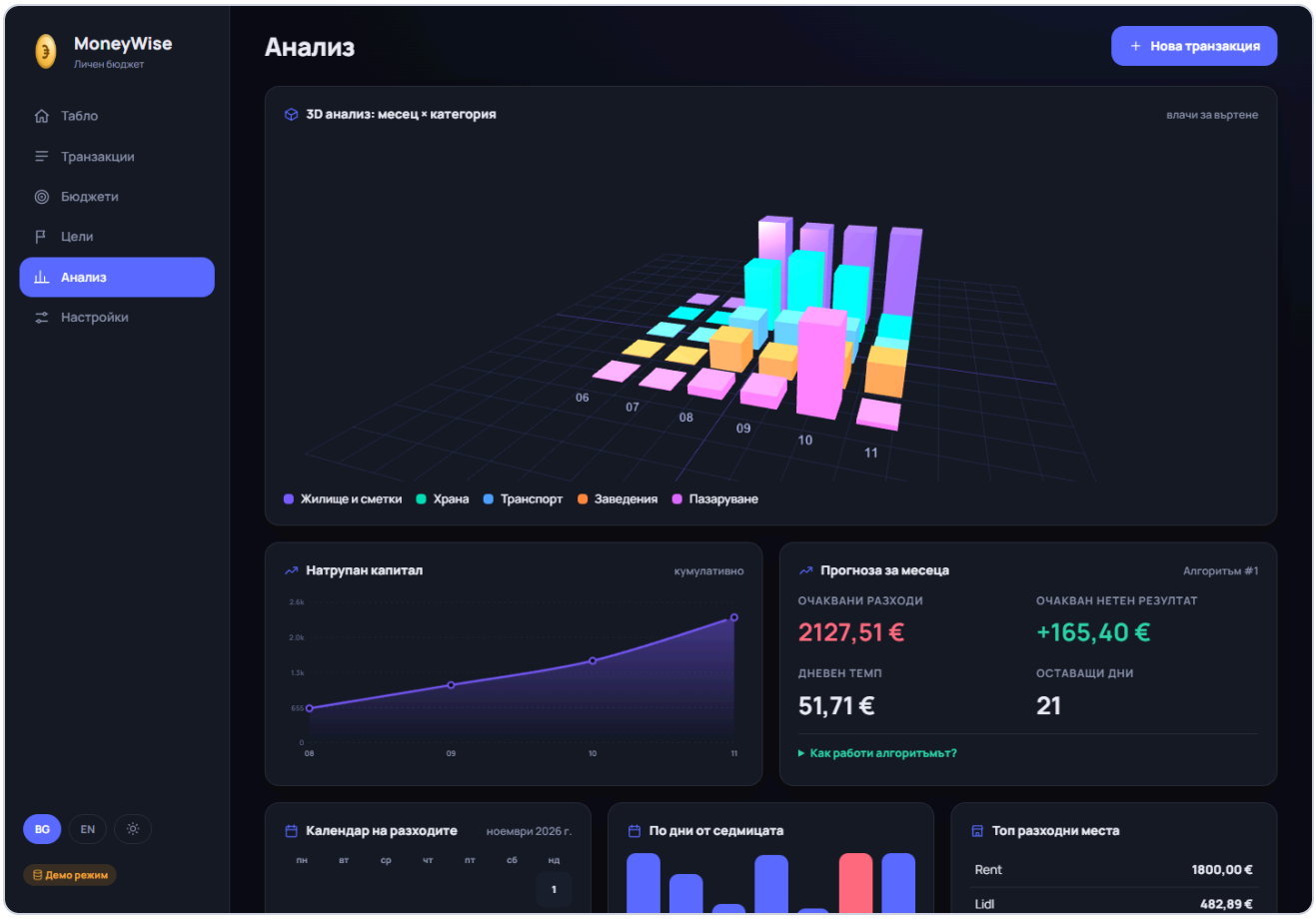
- **Транзакции** — списък по дни, търсене, филтри, експорт CSV:



- **Цели** — пръстени на прогреса, вноски, умно разпределение:

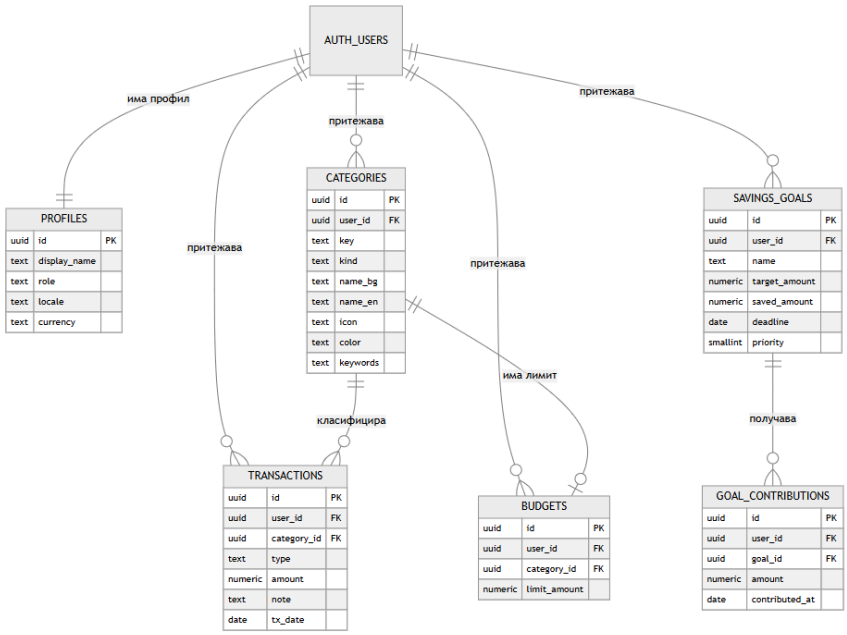


- **Анализ** — интерактивна 3D диаграма, прогноза, календар на разходите:




*Дизайнът е направен директно в код (HTML/CSS), а не във Figma; снимките са направени от работещата версия.*

# ER диаграма (начална и текуща)




## Описание на таблиците

| Таблица | Първичен ключ | Външни ключове | Ограничения |
|---|---|---|---|
| profiles | id (uuid) | id → auth.users | role и locale със CHECK, валута само EUR |
| categories | id | user_id → auth.users | kind IN (expense, income) |
| transactions | id | user_id → auth.users, category_id → categories (SET NULL) | amount > 0, type IN (...) |

| Таблица | Първичен ключ | Външни ключове | Ограничения |
|---|---|---|---|
| budgets | id | user_id, category_id → categories (CASCADE) | limit_amount > 0, UNIQUE(user_id, category_id) |
| savings_goals | id | user_id → auth.users | target_amount > 0, saved_amount ≥ 0, priority 1..3 |
| goal_contributions | id | user_id, goal_id → savings_goals (CASCADE) | amount > 0 |

**RLS:** на всички таблици политика „own rows“ — потребителят вижда и променя само редове със `user_id = auth.uid()` (за profiles: `id = auth.uid()`). Реализация: database/schema.sql.

# WEEK_1_REPORT — Етап 1

**Проект:** MoneyWise — личен бюджет в евро · **Екип:** Адам Биков и Реджеб Туджар · **Срок:** 14.11.2026

## 1. Какво планирахме за седмицата

- Проучване на три сходни решения (RESEARCH).
- План на функционалностите, технологиите и един собствен алгоритъм (PROJECT_PLAN).
- Поне два екрана за дизайн.
- Начална ER диаграма и таблици за Supabase.
- GitHub хранилище с README, структура на папките и поне шест Issues.
- По един собствен commit от всеки ученик.

## 2. Какво изпълнихме (конкретно)

| Дейност | Файл / екран | Отговорник |
|---|---|---|
| Проучване на 3 решения | `docs/RESEARCH.md` | Адам (Wallet, Spendee), Реджеб (YNAB, сравнителна таблица) |
| План, функции, технологии | `docs/PROJECT_PLAN.md` | двамата |
| Алгоритъм с псевдокод | `docs/PROJECT_PLAN.md` §2, `docs/ALGORITHM.md` | Реджеб |
| Потребителски истории | `docs/PRODUCT_SPEC.md` | Адам |
| Два екрана + допълнителни | `docs/SCREENS.md` , `docs/screens/*.png` | Адам |
| ER диаграма (6 таблици) | `docs/ER_DIAGRAM.md` , `docs/screens/er_diagram.png` | Реджеб |
| SQL схема с RLS | `database/schema.sql` | Реджеб |
| README и структура на папките | `README.md` , `src/` , `database/` , `docs/` , `tests/` | двамата |
| Работещ прототип (HTML/CSS/JS) | `src/index.html` , `src/css/style.css` , `src/js/*.js` | Адам (интерфейс), Реджеб ( `algo.js` , `store.js` ) |
| Автоматични тестове на алгоритъма | `tests/algo.test.js` (20 теста) | Реджеб |

| Дейност | Файл / екран | Отговорник |
|---|---|---|
| 15 задачи за Issues | `docs/ISSUES.md`, `scripts/create-issues.ps1` | двамата |

**Какво реално работи сега:** прототип, който се стартира локално ( `python -m http.server` в папка `src` ). Работят: табло, транзакции (добавяне, редакция, изтриване, търсене), бюджети, цели, анализ с диаграми, превключване BG/EN, светла/тъмна тема. Данните се пазят в браузъра (демо режим). **Връзката със Supabase е написана, но още не е тествана с реален проект.** Работеща публична версия още няма.

## 3. Какво не е готово и защо

| Задача | Причина |
|---|---|
| Supabase проект не е създаден; `schema.sql` не е изпълнен | предвидено за Етап 2; още нямаме проект и ключове |
| Връзката JS ↔ Supabase не е тествана | зависи от горното |
| GitHub Issues още не са създадени | отдалеченото хранилище не е свързано ( `origin` е със статус „gone“); трябва да се създаде и качи, после се пуска `scripts/create-issues.ps1` |
| Макети във Figma/Canva | дизайнът е направен директно в код; снимките са от работещата версия |
| Публикуване (GitHub Pages) | зависи от качване в GitHub |
| Ръчни тестове (docs/TESTS.md) | още не са проведени |

## 4. Следващи три задачи

1. Създаване на Supabase проект, изпълнение на `database/schema.sql`, проверка на RLS с два тестови акаунта — *Реджеб*.
2. Вход и регистрация и записване на транзакции в Supabase (JS ↔ база данни) — *Реджеб и Адам*.
3. Проверка на мобилния изглед и ръчни тестове, добавяне на липсващи екрани за грешки — *Адам*.

## 5. Използван изкуствен интелект

**Използвахме ИИ (Claude, Anthropic).** - **За какво:** генериране на първоначалния код на приложението, SQL схемата, документацията и тестовете; идеи за алгоритмите. - **Примерна заявка (prompt):** „MoneyWise — Управление на личен бюджет. Приложение за планиране и проследяване на личните финанси в евро ... създай нещо брутално реално и използваемо, с двуезичност BG/EN, Supabase и собствен алгоритъм; дизайнът да е креативен, с 3D елементи.“ - **Какво получихме:** работещ прототип (папка `src/` ), `database/schema.sql` , 20 теста и чернови на документите. - **Как проверихме и променихме:** -

стартирахме приложението и разгледахме всеки екран; - пуснахме `node tests/algo.test.js` — 20/20 успешни; - открихме и поправихме грешки: всички записи в „Последни транзакции“ се маркираха като „необичайно“; букви на български се изписваха с курсивни форми (т като „m“); предупрежденията за темп се показваха и за наема (фиксиран разход); - заменихме емоджитата с единен набор SVG икони и опростихме дизайна; - попълнихме имената и разпределихме задачите, пренаписахме проучването с конкретни наблюдения. - **Ограничение:** кодът е генериран с ИИ; всеки от нас трябва да може да обясни файловете си (вж. индивидуалните отчети). ☐ Допълнете какво още сте проверили и променили лично.

---

## Индивидуален отчет — Адам Биков (интерфейс)

**Какви задачи изпълних лично?** Проучих и описах Wallet и Spendee; написах потребителските истории; проектирах екраните (табло и бюджети) и описах елементите им; отговарях за интерфейса, навигацията, диаграмите, двуезичността и снимките на екраните.

**Кои файлове създадох?** `src/index.html`, `src/css/style.css`, `src/js/app.js`, `src/js/ui.js`, `src/js/charts.js`, `src/js/scene.js`, `src/js/icons.js`, `src/js/i18n.js`, `docs/PRODUCT_SPEC.md`, `docs/SCREENS.md`, `docs/screens/*.png`.

**Какъв проблем реших самостоятелно?** - Българските букви се изписваха странно (т като „m“, и като „u“) — причината е локализираната форма на шрифта за български; изключих я в `style.css` с `font-feature-settings: "locl" 0`. - В „Последни транзакции“ всичко се маркираше като „необичайно“, защото `map(txRow)` подаваше индекса като втори аргумент — поправих с `map(x => txRow(x))` в `app.js`. - Смених емоджитата с единен набор SVG икони (`icons.js`), за да изглежда интерфейсът цялостно.

**Какво научих?** Как се организира приложение без framework (модули, шаблони), как се прави двуезичност с един речник, как работят SVG диаграми и CSS 3D трансформации, как се чете проблем в браузъра (конзола, DevTools).

**Какво ще направя следващата седмица?** Ще проверя мобилния изглед и екраните за грешки, ще направя формите за вход/регистрация да работят със Supabase и ще проведа ръчните тестове от `docs/TESTS.md`.

---

## Индивидуален отчет — Реджеб Туджар (данни и алгоритъм)

**Какви задачи изпълних лично?** Проучих YNAB и написах сравнителната таблица; проектирах таблиците и ER диаграмата; написах SQL схемата с ограничения и RLS; описах и реализирах собствените алгоритми; написах автоматичните тестове.

**Кои файлове създадох?** `database/schema.sql`, `docs/ER_DIAGRAM.md`, `docs/screens/er_diagram.png`, `src/js/algo.js`, `src/js/store.js`, `tests/algo.test.js`, `docs/ALGORITHM.md`, `docs/TESTS.md`.

**Какъв проблем реших самостоятелно?** - Наемът (фиксиран разход) изкривяваше прогнозата за дневния темп — разделих разходите на фиксирани и променливи и ограничих еднократните големи покупки (winsorizing). - При постоянни суми (наем) откриването на аномалии даваше нулева разлика (MAD = 0) и

фалшиви тревоги — добавих резервна формула и изключих фиксираните категории. - Предупреждението „при този темп ще надхвърлиш" нямаше смисъл за наема — изключих го за фиксирани категории.

**Какво научих?** Как се проектира релационна база (първични и външни ключове, ограничения), какво е Row Level Security в PostgreSQL, как работят експоненциална пълзяща средна (EMA) и медиана/MAD за откриване на аномалии, как се пишат автоматични тестове с `assert`.

**Какво ще направя следващата седмица?** Ще създам Supabase проекта, ще изпълня `schema.sql`, ще проверя RLS с два тестови акаунта и ще довърша връзката `store.js` ↔ Supabase за всички таблици.